Jan-Øivind Lima

Procedural generation: music

Generating music using method X and Y

Master's thesis in Cybernetics and Robotics
Supervisor: Sverre Hendseth
June 2026

Norwegian University of Science and Technology
Faculty of Information Technology and Electrical Engineering
Department of Engineering Cybernetics

NTNU

ABSTRACT

Write an abstract/summary of your thesis, and state your main findings here.

A summary should be included in both English and any second language, if this is applicable, regardless if the thesis is written in English or in your preferred language. These should be on separate pages, the English version first.

PREFACE

Write the preface of your thesis here.

You may include acknowledgements and thanks as part of your preface on this page, or you may add it as a new chapter after the preface.

CONTENTS

Abstract **i**

Preface **ii**

Contents **iv**

List of Figures **iv**

List of Tables **v**

Abbreviations **vii**

1 Introduction **1**
1.1 Context and Motivation 1
1.2 Problem description 1
1.3 Problem solution 1

2 Theory **3**
2.1 Music theory 3
2.1.1 Notes 3
2.1.2 Scales 3
2.1.3 Keys 4
2.1.4 Melodies 4
2.1.5 Harmony 4
2.2 Procedural generation 5
2.2.1 Wave function collapse 5
2.3 Object oriented programming 5

3 Software design **7**

4 Implementation **9**

5 Results **11**
5.1 First iteration 11
5.1.1 Features 11
5.1.2 Limitations 11
5.1.3 Specifications for the second iteration 12

6 Discussion **13**
6.1 Future work 13

7 Conclusions **15**

References **17**

Appendices: **19**

A - Github repository **20**

B - Audio examples **21**

C - Code documentation **22**

LIST OF FIGURES

2.1.1 The c-major scale written in the treble clef. 4
2.1.2 The eb-major scale written in the treble clef. 4
2.1.3 Harmonic progression in tonal music. The arrows indicate the typical flow of harmonic functions. 5

5.1.1 Example melody of the first iteration melody generator. 11

LIST OF TABLES

ABBREVIATIONS

List of all abbreviations in alphabetic order:

- **NTNU** Norwegian University of Science and Technology

CHAPTER

ONE

INTRODUCTION

For hundreds of years composers have been creating music using external tools like dice to help them generate something new. Creating a good solution for generating music procedurally has yet to be done and this thesis will explore a way to solve this problem.

1.1 Context and Motivation

Creating a good solution for generating good music will not just be applicable to generating music, but the structure will be able to be used for a number of other applications. That could be for other artistic purposes or any application where generating something that cannot be completely random.

1.2 Problem description

Project decs. bla bla

1.3 Problem solution

CHAPTER

TWO

THEORY

In order to understand how melodies are structured we need to lay some theoretical groundwork using music theory. Additionally we will cover some concepts regarding procedural generation, *this can include wave function collapse, or other things researched.*

2.1 Music theory

2.1.1 Notes

The very core of music is composed of notes. Notes are vibration in air which we interpret as pitches. In the western system we agree upon the standard of

2.1.2 Scales

The musical scale is building block for creating not only melodies but also harmony, which we will cover a bit later. Firstly the musical scale is, for the most part, constructed using seven notes. In the western canon this is usually seven notes. You can see the basic c-major scale in Figure 2.1.1, it is comprised of the notes c d e f g a b c. The tones of the scale can be named according to their function within the scale. We often label the notes $\hat{1}, \hat{2}, \hat{3}, \hat{4}, \hat{5}, \hat{6}, \hat{7}$, and then starting back at $\hat{1}$. The use of the carets are called the Nashville Number System, this is used to differentiate the scale degrees from the chord tones which will be discussed in Section 2.1.5 When we consider chord structures we use the numbers but usually count above from 8 to signify harmonic functions, but when considering only scales and melodies we usually only use 1 through 7

The most important function is that of the scale degree 1, this is the *tonic*, which is the 'home' of the key. This is the most stable function there is, it feels resolved and does not have any tension urging it to move anywhere.

Next up we have the dominant function, there are two notes degrees which fall into this category, that is the 5th and 7th scale degree. These notes have tension. They want to move, especially the 7th scale degree, it has a great pull toward the tonic. You can hear this in the example under Figure 2.1.1. The penultimate tone is the 7th scale degree and it leads to the tonic.

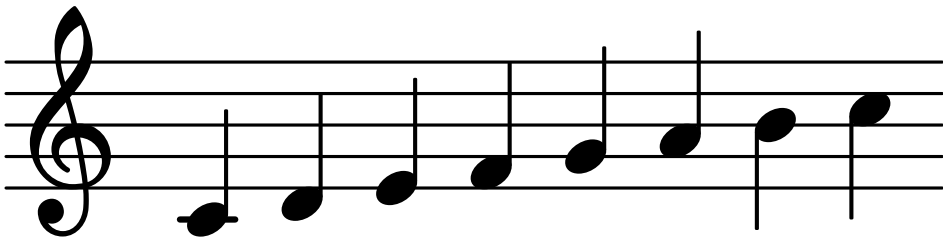

Figure 2.1.1: The c-major scale written in the treble clef.

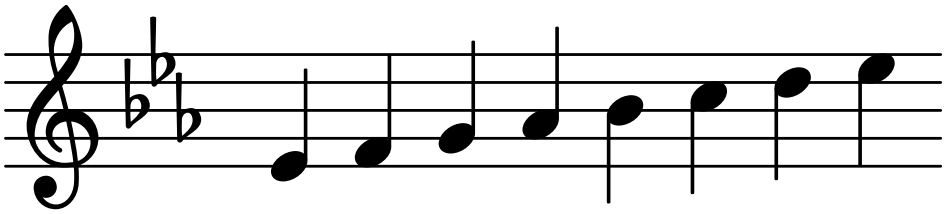

Figure 2.1.2: The eb-major scale written in the treble clef.

To lead us to the tonic functions we have what are called the pre-dominant function. This simply means 'before dominant', and their function is to set up the tension to lead us to the dominant degrees which in turn leads us back to the tonic.

2.1.3 Keys

The concept of a musical key is simply shifting around where the tonic note is. For instance in Figure 2.1.1 you can see

2.1.4 Melodies

2.1.5 Harmony

When considering the harmonic progression in tonal music we can consider all the standard chords in the major key. They are labeled by numbers using the roman numerals. Upper case means the chord in major and lower case means minor. You can see the difference between major and minor chords in figure [make a figure for this], and listen to the difference at the link provided in the caption.

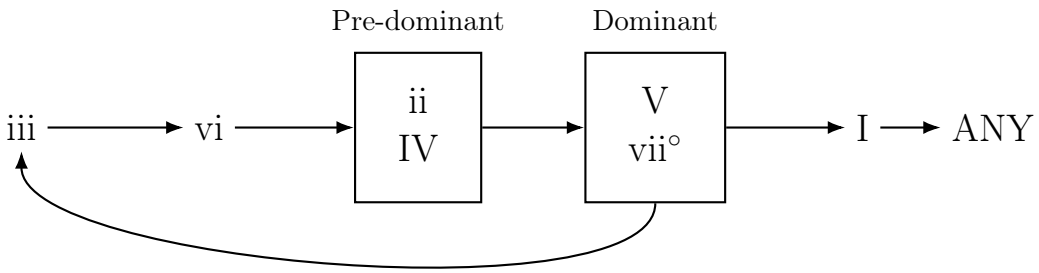

Figure 2.1.3: Harmonic progression in tonal music. The arrows indicate the typical flow of harmonic functions.

2.2 Procedural generation

This section covers the relevant theory regarding procedural generation.

2.2.1 Wave function collapse

Might use this in the proc-gen so a chapter on this might be useful.

2.3 Object oriented programming

[1]

CHAPTER

THREE

SOFTWARE DESIGN

Include the complete description of the methods used in your research here.

CHAPTER

FOUR

IMPLEMENTATION

The implementation of the solution will be described in this chapter. The implementation will be done in Python, and the code will be available on GitHub. The implementation will be divided into three main parts: the data collection, the model training, and the music generation. Each part will be described in detail in the following sections.

CHAPTER

FIVE

RESULTS

5.1 First iteration

After a few weeks of development the project grinded to a halt due to the framework established was incompatible with the ideas of how to improve the system. This led to the first refactor of the project. The code was working, but unable to be improved upon. I took what I learned from this first work and started planning the next iteration using the limitations of the first iteration as a spec for the next one.

The first iteration had some good features, but also lacked several important ones I found to be important after running into the first wall of the project.

5.1.1 Features

The system was able to generate random melodies using stepwise motion, meaning moving by 2nds, and skips, meaning moving by 3rds. This made for somewhat singable melodies.

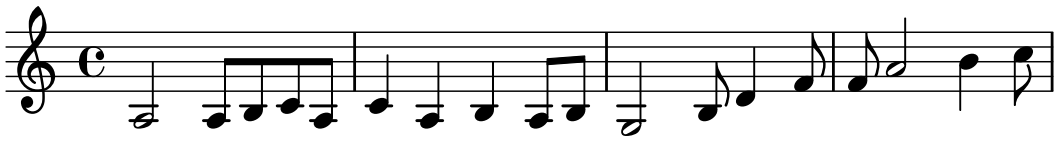

Figure 5.1.1: Example melody of the first iteration melody generator.

5.1.2 Limitations

The system had no way to track which octave it was in which meant that longer melodies had a tendency to meader outside what would be considered as singable or even playable register.

There were no consideration of tension and release in the melody. Phrases never

Rests were not implemented in any way, pauses and rests are an important part of a melody and this needs to be addressed so that the melodies are not simply a continous stream of notes.

5.1.3 Specifications for the second iteration

For the second iteration I wanted to focus on these things.

- No absolute notes until rendering(keep it at scale degrees, to reduce overhead)
- Octave tracking
- Restricted range
- Ending prases resolved(tonic, long note)
- Self similarity

CHAPTER

SIX

DISCUSSION

Discuss your results here.

6.1 Future work

Include a section about what should or could be done in future research, or explain any recommended next steps based on the results you got. This should be the last section in the discussion.

CHAPTER

SEVEN

CONCLUSIONS

Give a concise summary of your research and finding here, and include a short summary of any future work as well.
Looking at this thing here, [2] [3] [4] it is a citation.

REFERENCES

[1] Ivar Jacobson et al. *Object-Oriented Software Engineering*. Addison-Wesley, 1992. ISBN: 0-201-54435-0.

[2] Roland van der Linden, Ricardo Lopes, and Rafael Bidarra. "Procedural Generation of Dungeons". In: *IEEE Transactions on Computational Intelligence and AI in Games* 6.1 (Mar. 2014), pp. 78–89. ISSN: 1943-0698. DOI: 10.1109/TCIAIG.2013.2290371. URL: https://ieeexplore.ieee.org/abstract/document/6661386 (visited on 02/26/2026).

[3] E. Galin et al. "Procedural Generation of Roads". In: *Computer Graphics Forum* 29.2 (2010), pp. 429–438. ISSN: 1467-8659. DOI: 10.1111/j.1467-8659.2009.01612.x. URL: https://onlinelibrary.wiley.com/doi/abs/10.1111/j.1467-8659.2009.01612.x (visited on 02/26/2026).

[4] Johan Gangsås Hole. "Automatic Species Counterpoint". In: *Master Thesis at NTNU* (2026).

APPENDICES

A - GITHUB REPOSITORY

All code and latex-files used in this document are included in the Github repository linked below. Further explanations are given in the readme-file.

Github repository link

- `https://github.com/lima98/TTK4900`

B - AUDIO EXAMPLES

Audio-files for the examples in the thesis can be played from my NTNU-webpage.

Audio example archive

- `https://www.janoivil.folk.ntnu.no`

C - CODE DOCUMENTATION

All documentation for the code is included in the Github repository linked in 7 it is also provided as a web-page hosted at my NTNU-webpage.

Code documentation

- `https://janoivil.folk.ntnu.no/docs/_build/html/index.html`